

UNIVERSIDADE REGIONAL DO NOROESTE DO
ESTADO DO RIO GRANDE DO SUL
CURSO DE CIÊNCIA DA COMPUTAÇÃO

JEAN KRONBAUER DE MOURA

DEPLOYDOCK: ARQUITETURA, IMPLEMENTAÇÃO E AVALIAÇÃO EMPÍRICA DE UMA PLATAFORMA COMO SERVIÇO (PAAS) AUTOHOSPEDADA COM FOCO EM HARDENING DE CONTÊINERES E GITOPS

Ijuí/RS
2026

JEAN KRONBAUER DE MOURA

DEPLOYDOCK: ARQUITETURA, IMPLEMENTAÇÃO E AVALIAÇÃO EMPÍRICA DE UMA PLATAFORMA COMO SERVIÇO (PAAS) AUTOHOSPEDADA COM FOCO EM HARDENING DE CONTÊINERES E GITOPS

Trabalho de Conclusão de Curso apresentado ao Curso de Ciência da Computação da Universidade Regional do Noroeste do Estado do Rio Grande do Sul - UNIJUÍ, como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação.

Orientador(a): Prof. Marcos Ronaldo Melo Cavalheiro

Ijuí/RS

2026

JEAN KRONBAUER DE MOURA

DEPLOYDOCK: ARQUITETURA, IMPLEMENTAÇÃO E AVALIAÇÃO EMPÍRICA DE UMA PLATAFORMA COMO SERVIÇO (PAAS) AUTOHOSPEDADA COM FOCO EM HARDENING DE CONTÊINERES E GITOPS

Trabalho de Conclusão de Curso apresentado ao Curso de Ciência da Computação da Universidade Regional do Noroeste do Estado do Rio Grande do Sul - UNIJUÍ, como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação, submetido à aprovação da banca examinadora composta pelos seguintes membros:

Prof. Marcos Ronaldo Melo Cavalheiro
Orientador(a)

Prof. Marcos Ronaldo Melo Cavalheiro
Membro da Banca

Ijuí/RS, 03 de Nov de 2026

AGRADECIMENTOS

Agradeço primeiramente a Deus, pela dádiva da vida, saúde, força e clareza mental que me sustentaram ao longo de toda esta jornada acadêmica.

Aos meus pais e familiares, pelo amor incondicional, apoio irrestrito, sacrifícios diários e por acreditarem no meu potencial em cada etapa desta caminhada. A compreensão e o incentivo de vocês foram essenciais para transformar este sonho em realidade.

Ao meu orientador, Prof. Marcos Ronaldo Melo Cavalheiro, pela dedicação, confiança, valiosas orientações técnicas e acadêmicas, paciência e pelo compartilhamento generoso de conhecimento que nortearam o amadurecimento desta pesquisa.

Aos professores do Curso de Ciência da Computação da Universidade Regional do Noroeste do Estado do Rio Grande do Sul (UNIJUÍ), que com zelo e excelência contribuíram para minha sólida formação profissional, despertando a paixão pela investigação científica e pela computação aplicada.

Aos colegas de curso e amigos de jornada, pelos momentos de companheirismo, noites em claro resolvendo desafios de código, discussões enriquecedoras e pela amizade fraterna construída ao longo destes anos.

Por fim, a toda a comunidade de software livre e de código aberto, cujo espírito de cooperação global disponibiliza ferramentas indispensáveis para a democratização e o progresso da tecnologia.

“Simplicity is prerequisite for reliability.”
(Edsger W. Dijkstra)

LISTA DE ILUSTRAÇÕES

Figura 1 – Painel Inicial do DeployDock com Visão Geral do Sistema 28
Figura 2 – Módulo de Telemetria e Monitoramento de Recursos em Tempo Real 28
Figura 3 – Catálogo de Aplicações e Projetos Gerenciados 29
Figura 4 – Histórico de Deploys, Inspeção de Logs e Rollback 29
Figura 5 – Gerenciamento do Ciclo de Vida de Contêineres e Terminal 30
Figura 6 – Central de Segurança Ativa e Eventos do WAF 30
Figura 7 – Central de Bancos de Dados SQL Gerenciados (PostgreSQL 16) 31
Figura 8 – Interface da Sonda Autônoma de Auditoria e Benchmark Científico 31

LISTA DE QUADROS

Quadro 1 – Matriz Comparativa Multidimensional entre Plataformas PaaS Abertas . . 35

LISTA DE TABELAS

Tabela 1 – Distribuição e Taxa de Sucesso dos 58 Testes E2E Auditados 32
Tabela 2 – Métricas Empíricas de Estresse Computacional nos Dois Ambientes de Teste 34

LISTA DE ABREVIATURAS E SIGLAS

ACID	Atomicidade, Consistência, Isolamento e Durabilidade (*Atomicity, Consistency, Isolation, Durability*)
AES	Padrão Avançado de Criptografia (*Advanced Encryption Standard*)
API	Interface de Programação de Aplicações (*Application Programming Interface*)
ASGI	Interface de Gateway de Servidor Assíncrono (*Asynchronous Server Gateway Interface*)
CI/CD	Integração Contínua e Entrega Contínua (*Continuous Integration / Continuous Delivery*)
CPU	Unidade Central de Processamento (*Central Processing Unit*)
DoS	Negação de Serviço (*Denial of Service*)
DSR	Pesquisa em Ciência do Design (*Design Science Research*)
E2E	Ponta a Ponta (*End-to-End*)
HMAC	Código de Autenticação de Mensagens Baseado em Hash (*Hash-based Message Authentication Code*)
HTTP	Protocolo de Transferência de Hipertexto (*Hypertext Transfer Protocol*)
HTTPS	Protocolo de Transferência de Hipertexto Seguro (*Hypertext Transfer Protocol Secure*)
IOPS	Operações de Entrada/Saída por Segundo (*Input/Output Operations Per Second*)
JWT	Token Web JSON (*JSON Web Token*)
MFLOPS	Milhões de Operações de Ponto Flutuante por Segundo (*Million Floating-Point Operations Per Second*)
OSI	Interconexão de Sistemas Abertos (*Open Systems Interconnection*)
OWASP	Projeto Aberto de Segurança em Aplicações Web (*Open Worldwide Application Security Project*)
PaaS	Plataforma como Serviço (*Platform as a Service*)

PAT	Token de Acesso Pessoal (*Personal Access Token*)
PID	Identificador de Processo (*Process Identifier*)
RAM	Memória de Acesso Aleatório (*Random Access Memory*)
RBAC	Controle de Acesso Baseado em Papéis (*Role-Based Access Control*)
RCE	Execução Remota de Código (*Remote Code Execution*)
SGBDR	Sistema de Gerenciamento de Banco de Dados Relacional
SHA	Algoritmo de Hash Seguro (*Secure Hash Algorithm*)
SQL	Linguagem de Consulta Estruturada (*Structured Query Language*)
SQLi	Injeção de SQL (*SQL Injection*)
SSRF	Falsificação de Requisição do Lado do Servidor (*Server-Side Request Forgery*)
TLS	Segurança da Camada de Transporte (*Transport Layer Security*)
TOTP	Senha Descartável Baseada em Tempo (*Time-Based One-Time Password*)
TPS	Transações por Segundo (*Transactions Per Second*)
TTFB	Tempo até o Primeiro Byte (*Time To First Byte*)
WAF	Firewall de Aplicações Web (*Web Application Firewall*)
WAL	Log de Pré-Gravação (*Write-Ahead Logging*)
XSS	Scripting em Múltiplos Sites (*Cross-Site Scripting*)

SUMÁRIO

APRESENTAÇÃO **12**
RESUMO **13**
ABSTRACT **13**
INTRODUÇÃO **15**
SOLUÇÃO PROPOSTA: DEPLOYDOCK 16
OBJETIVOS DA PESQUISA 17
ESTRUTURA DO DOCUMENTO 17
METODOLOGIA **18**
ETAPAS DO PROCESSO METODOLÓGICO 18
USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA NA IMPLEMENTAÇÃO DO PROJETO 19
DELINEAMENTO EXPERIMENTAL E PROCEDIMENTOS DE TESTE 20
AMBIENTES DE TESTE E INFRAESTRUTURA 20
REFERENCIAL TEÓRICO **21**
VIRTUALIZAÇÃO NO NÍVEL DO SISTEMA OPERACIONAL E CONTÊINERES . . 21
PLATAFORMA COMO SERVIÇO (PAAS) E FILOSOFIA GITOPS 22
SEGURANÇA DEFENSIVA EM APLICAÇÕES WEB E HARDENING DE CONTÊINERES 23
INTEGRIDADE TRANSACIONAL ACID EM BANCOS DE DADOS RELACIONAIS 24
DESENVOLVIMENTO E AVALIAÇÃO EMPÍRICA **24**
ARQUITETURA GERAL DO DEPLOYDOCK 24
CAMADA DE SEGURANÇA ATIVA E WAF (SECURITY SHIELD) 25
ORQUESTRADOR GITOPS COM AUTO-DOCKERFILE E ROLLBACK INSTANTÂNEO 26
TERMINAL WEB PROTEGIDO E AJUSTE DINÂMICO DE QUOTAS 27
APRESENTAÇÃO DA INTERFACE DO USUÁRIO 27
RESULTADOS E DISCUSSÃO 32
Validação Funcional e de Segurança E2E (58 Testes) **32**
Validação Científica das Propriedades ACID no PostgreSQL 16 . . . . . **33**
Benchmark de Estresse e Desempenho de Máquina (Intensidade Heavy) **33**
Matriz Comparativa com Plataformas Correlatas (PaaS Benchmark) **34**
CONSIDERAÇÕES FINAIS **35**
CONTRIBUIÇÕES DA PESQUISA 36
LIMITAÇÕES IDENTIFICADAS 36
TRABALHOS FUTUROS 37

REFERÊNCIAS **38**

ESTRUTURA DO ESQUEMA DE DADOS DO LAUDO DE AUDITORIA (JSON) . 40
CRITÉRIOS DE PONDERAÇÃO DO CATÁLOGO DE FUNCIONALIDADES PAAS 42
CATÁLOGO DOS CASOS DE TESTE E2E DA SONDA DE AUDITORIA 43

APRESENTAÇÃO

O presente Trabalho de Conclusão de Curso, intitulado **"DEPLOYDOCK: ARQUITETURA, IMPLEMENTAÇÃO E AVALIAÇÃO EMPÍRICA DE UMA PLATAFORMA COMO SERVIÇO (PaaS) AUTOHOSPEDADA COM FOCO EM HARDENING DE CONTÊINERES E GITOPS"**, é submetido ao Colegiado do Curso de Ciência da Computação da Universidade Regional do Noroeste do Estado do Rio Grande do Sul (UNIJUÍ) como requisito para a obtenção do grau de Bacharel em Ciência da Computação. O documento organiza-se no formato de Artigo Científico estruturado segundo as normas da ABNT e as diretrizes do manual institucional da universidade.

A pesquisa investiga os desafios de sustentabilidade operacional, orquestração enxuta e segurança cibernética em infraestruturas autohospedadas, articulando a teoria de microsserviços e sistemas operacionais a um estudo empírico com baterias ponta a ponta e testes de estresse de hardware. Após as páginas pré-textuais, sucedem-se a Introdução, a Metodologia, o Referencial Teórico, o Desenvolvimento e Resultados, as Considerações Finais e os elementos pós-textuais (declaração de integridade no uso de ferramentas de IA, referências bibliográficas, apêndice e anexos comprobatórios).

DEPLOYDOCK: ARQUITETURA, IMPLEMENTAÇÃO E AVALIAÇÃO EMPÍRICA DE UMA PLATAFORMA COMO SERVIÇO (PAAS) AUTOHOSPEDADA COM FOCO EM HARDENING DE CONTÊINERES E GITOPS

DEPLOYDOCK: ARCHITECTURE, IMPLEMENTATION AND EMPIRICAL EVALUATION OF A SELF-HOSTED PLATFORM AS A SERVICE (PAAS) FOCUSED ON CONTAINER HARDENING AND GITOPS

Jean Kronbauer de Moura [1]

Marcos Ronaldo Melo Cavalheiro [2]

RESUMO

A virtualização orientada a contêineres e a cultura DevOps revolucionaram o ciclo de desenvolvimento de software, tornando essencial a adoção de esteiras automatizadas de entrega contínua. Contudo, orquestradores como o Kubernetes apresentam elevada complexidade e sobrecarga computacional para servidores modestos e aplicações autohospedadas, enquanto plataformas PaaS de código aberto existentes frequentemente negligenciam proteções na camada de aplicação e o isolamento de privilégios em contêineres. Este trabalho apresenta o DeployDock, uma Plataforma como Serviço (PaaS) autohospedada, leve e orientada a GitOps desenvolvida em Python assíncrono (FastAPI), Docker SDK e proxy reverso Traefik v3. A solução incorpora um Web Application Firewall (WAF) ativo perimetral, cofre de variáveis de ambiente com criptografia Fernet (AES-128), ajuste dinâmico de quotas de CPU/RAM em tempo de execução sem interrupção de contêineres e rollback automatizado em falhas. Para validação experimental, construiu-se uma sonda autônoma independente submetendo a plataforma a 58 testes E2E, estresse multivariado de hardware e 11 testes transacionais no PostgreSQL 16. Os resultados comprovaram 100% de aprovação funcional e bloqueio defensivo de ataques de SQLi, XSS, Path Traversal e RCE, score de hardening de 95/100 sob Linux corporativo e cumprimento rigoroso das propriedades ACID sob vazão superior a 3.200 inserções/s, operando sob uma pegada basal de memória de apenas 79,2 MB em repouso.

Palavras-chave: computação em nuvem; plataforma como serviço; docker; orquestração de contêineres; segurança da informação; gitops.

---

[1] Estudante do Curso de Ciência da Computação – UNIJUÍ, Ijuí-RS, Brasil. E-mail: jean.moura@sou.unijui.edu.br

[2] Professor Orientador – UNIJUÍ, Ijuí-RS, Brasil. E-mail: marcos.cavalheiro@unijui.edu.br

ABSTRACT

Container-based virtualization and DevOps practices have deeply transformed the software development lifecycle, making continuous delivery pipelines paramount. However, complex orchestrators such as Kubernetes impose excessive cognitive and resource overhead for small-to-medium servers and self-hosted deployments, whereas existing open-source PaaS alternatives often lack integrated application-layer firewalls and strict container privilege isolation. This study presents DeployDock, a lightweight, self-hosted, GitOps-oriented Platform as a Service (PaaS) built with asynchronous Python (FastAPI), the Docker SDK, and Traefik v3 reverse proxy. The platform embeds an active Web Application Firewall (WAF) middleware, secret storage encryption via Fernet (AES-128), live dynamic CPU/RAM cgroup quota tuning without container downtime, and instant automated rollback upon health check failures. For empirical validation, an independent autonomous probe evaluated DeployDock against 58 E2E assertions, heavy multi-core hardware stress benchmarks, and 11 transactional tests targeting PostgreSQL 16. Experimental findings demonstrated a 100% pass rate across all functional suites, robust perimeter mitigation against SQLi, XSS, Path Traversal, and RCE vectors, a 95/100 container hardening score under enterprise Linux, and strict compliance with ACID guarantees under bulk insertion rates exceeding 3,200 rows/second, all while maintaining a remarkably low memory footprint of only 79.2 MB at idle.

Keywords: cloud computing; platform as a service; docker; container orchestration; information security; gitops.

INTRODUÇÃO

A consolidação da computação em nuvem e a transição de monólitos de software para arquiteturas baseadas em microsserviços transformaram radicalmente as etapas de desenvolvimento, teste e implantação de sistemas computacionais. No centro dessa transformação, a virtualização baseada em contêineres — especialmente capitaneada pelo ecossistema Docker — estabeleceu-se como padrão *de facto* na indústria de software (Merkel, 2014). Ao empacotar a aplicação com todas as suas dependências e bibliotecas em unidades atômicas e portáveis, os contêineres garantem previsibilidade e eliminam discrepâncias entre ambientes de homologação e produção (Combes e Petazzoni, 2016).

No entanto, o gerenciamento do ciclo de vida dessas aplicações conteinerizadas introduz uma camada substancial de complexidade operacional. Práticas modernas como a Entrega Contínua (*Continuous Delivery*) (Humble e Farley, 2010) e o paradigma GitOps — no qual repositórios de controle de versão atuam como a fonte única da verdade para o estado desejado da infraestrutura — exigem esteiras automatizadas de construção (*build*), provisionamento de certificados digitais, roteamento reverso de tráfego de rede e monitoramento de integridade (*health check*).

Para atender a essas demandas, o mercado corporativo tradicionalmente recorre a orquestradores de contêineres em larga escala, dos quais o Kubernetes é o expoente mais difundido (Burns et al., 2016). Embora o Kubernetes apresente capacidades sólidas de escalabilidade horizontal e resiliência em centros de dados distribuídos, sua adoção impõe uma sobrecarga computacional acentuada e uma curva de aprendizado íngreme. A complexidade de configuração baseada em centenas de manifestos YAML, o consumo excessivo de memória em repouso dos planos de controle (*control plane*) e a necessidade de equipe especializada inviabilizam sua sustentabilidade econômica e técnica em infraestruturas modestas, pequenos servidores autohospedados (*bare-metal* ou VPS), laboratórios acadêmicos e pequenas empresas de base tecnológica.

Como alternativa, consolidaram-se as Plataformas como Serviço (*Platform as a Service* – PaaS), tanto em ofertas proprietárias sob demanda na nuvem pública (tais como Heroku, Render e Railway), quanto em soluções de código aberto autohospedadas (*self-hosted*), exemplificadas por CapRover, Dokploy e Coolify. Não obstante a evolução das ferramentas de código aberto, uma investigação aprofundada de suas arquiteturas revela lacunas críticas:

(a) **Vulnerabilidade na Camada de Aplicação:** a grande maioria dos orquestradores PaaS foca exclusivamente no roteamento HTTP (frequentemente via Traefik ou Nginx), transferindo integralmente a responsabilidade de mitigação de ataques à aplicação, carecendo de um Web Application Firewall (WAF) ativo e integrado capaz de neutralizar vetores como Injeção de SQL (*SQL Injection*), *Cross-Site Scripting* (XSS), *Path Traversal* e Execução Remota de Código (RCE) antes que atinjam o núcleo do sistema;

(b) **Falta de Hardening Padrão no Contêiner:** por conveniência operacional, muitos contêineres gerenciados são criados sem imposição de restrições de privilégios (‘no-new-privileges‘), sem cotas estritas de criação de processos (‘pids.max‘), permitindo que falhas de código ou bibliotecas comprometidas provoquem esgotamento de recursos no hospedeiro (*fork bomb*) ou escalação de privilégios via binários com bits SUID;

(c) **Gestão Rígida de Recursos:** o ajuste de quotas de memória RAM e núcleos de CPU geralmente demanda a interrupção completa e recriação do contêiner, interrompendo a prestação de serviços;

(d) **Ausência de Baterias de Auditoria Científica:** inexistem mecanismos integrados nas plataformas que permitam auditar e mensurar, de forma empírica e reprodutível, a conformidade de transações de bancos de dados relacionais com as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade) (Stonebraker e Rowe, 1986), bem como limites de estresse de CPU, memória, I/O e rede.

Diante dessa problemática, emerge a seguinte questão de pesquisa: *Como conceber, implementar e avaliar empiricamente uma Plataforma como Serviço (PaaS) autohospedada, leve e orientada a GitOps, que integre nativamente mecanismos ativos de defesa (WAF), hardening estrito de contêineres e orquestração de bancos de dados relacionais sob garantias ACID?*

SOLUÇÃO PROPOSTA: DEPLOYDOCK

Para responder a essa questão, este trabalho apresenta o projeto e a concretização do **DeployDock**, um ecossistema PaaS de código aberto desenvolvido sobre a linguagem Python utilizando o framework assíncrono FastAPI, SQLAlchemy Assíncrono, orquestração via Docker SDK e roteamento dinâmico baseado em Traefik v3. O DeployDock incorpora uma camada ativa de segurança (*Security Shield*) que intercepta tráfego HTTP malicioso em nível de middleware, criptografa segredos em repouso por meio do algoritmo Fernet (AES-128 em modo CBC com autenticação HMAC-SHA256), aplica regras de controle de processos via *cgroups* do Linux (com limites automáticos de PIDs e bloqueio de novos privilégios) e suporta atualização dinâmica de quotas de recursos em tempo de execução sem interrupção de contêineres.

Além da infraestrutura de deploy, o projeto engloba o desenvolvimento de uma sonda autônoma de instrumentação e testes denominada *DeployDock Multi-Model Benchmark & PaaS Tester Probe v2.2*. Esta ferramenta foi especificamente modelada para submeter a plataforma a uma bateria exaustiva de 58 testes funcionais de ponta a ponta (E2E), testes transacionais direcionados ao PostgreSQL 16 e rotinas de estresse computacional sob condições extremas de hardware.

OBJETIVOS DA PESQUISA

O **objetivo geral** deste trabalho de conclusão de curso consiste em projetar, implementar e validar empiricamente uma Plataforma como Serviço (PaaS) autohospedada e segura, denominada DeployDock, avaliando seu desempenho, eficácia defensiva e conformidade arquitetural frente aos desafios operacionais de microsserviços.

Como **objetivos específicos**, definem-se:

- Modelar e construir a arquitetura do DeployDock em microframework assíncrono, integrando roteamento dinâmico via Traefik v3, emissão automatizada de certificados TLS Let's Encrypt com wildcard e cofre criptográfico para variáveis de ambiente;
- Desenvolver um orquestrador GitOps com suporte a acionamento via *webhooks*, detecção inteligente e automatizada de stacks tecnológicas (*Zero-Config / Auto-Dockerfile*) e estratégia de *health check* com reversão instantânea (*rollback*) em falhas;
- Implementar o módulo de segurança defensiva ativa com WAF embutido, limitador de taxa (*rate limiting*), bloqueio de força bruta, controle de acesso baseado em papéis (RBAC) e *hardening* padrão de contêineres;
- Projetar a suíte de auditoria e a sonda empírica independente para execução de 58 testes E2E, estresse multivariado de hardware e validação rigorosa das propriedades ACID no banco de dados gerenciado PostgreSQL 16;
- Analisar quantitativamente os resultados de latência, throughput, isolamento de processos e consumo de memória em repouso, contrastando os achados com soluções correlatas do estado da arte.

ESTRUTURA DO DOCUMENTO

O restante deste trabalho está estruturado da seguinte forma: A seção 2 descreve o delineamento metodológico da pesquisa, caracterizando a abordagem experimental adotada, as variáveis mensuradas e os ambientes operacionais de teste. A seção 3 estabelece os fundamentos teóricos em virtualização por contêineres, orquestração PaaS, princípios GitOps, segurança em aplicações Web e integridade transacional ACID. A seção 4 detalha a arquitetura do DeployDock, suas decisões de implementação de software, a instrumentação da sonda de testes e discute aprofundadamente os dados empíricos obtidos na validação funcional, no teste de estresse de máquina e na comparação de mercado. Por fim, a seção 5 sintetiza as principais conclusões, destacando contribuições, limitações e perspectivas de trabalhos futuros.

METODOLOGIA

Para responder à questão de pesquisa e concretizar os objetivos propostos, este trabalho adota uma abordagem de **pesquisa aplicada**, com natureza predominantemente **quantitativa e experimental**, estruturada metodologicamente sob as diretrizes da *Design Science Research* (DSR). Conforme preceituado na literatura clássica de Engenharia de Software (Pressman e Maxim, 2016; Sommerville, 2011), a DSR fornece um referencial sistemático para projetar, construir e avaliar artefatos computacionais que apresentem soluções inovadoras e eficazes para problemas operacionais reais.

Nesta pesquisa, o artefato primário desenvolvido é a plataforma PaaS **DeployDock**, enquanto o artefato instrumental de avaliação é a sonda autônoma **DeployDock Multi-Model Benchmark & PaaS Tester Probe v2.2**.

ETAPAS DO PROCESSO METODOLÓGICO

O ciclo de vida metodológico foi estruturado em cinco fases iterativas:

1. **Fase 1: Conscientização do Problema e Levantamento de Requisitos:** Identificação das lacunas de segurança (ausência de WAF nativo), vulnerabilidades de contêineres (falta de limites de PIDs e ausência de bloqueio de novos privilégios) e sobrecarga de memória presentes nas soluções PaaS de código aberto consolidadas (CapRover, Coolify v4 e Dokploy);

2. **Fase 2: Concepção Arquitetural e Modelagem Teórica:** Definição do modelo em camadas, especificando a separação entre a API REST assíncrona, o motor de eventos WebSockets, o proxy reverso Traefik v3 com suporte a certificados curinga e o cofre criptográfico para armazenamento de segredos;

3. **Fase 3: Engenharia e Implementação de Software com Suporte de Inteligência Artificial:** Codificação modular do núcleo do sistema em Python assíncrono (FastAPI), utilizando agentes e assistentes de Inteligência Artificial Generativa para apoiar a síntese de algoritmos defensivos, padrões de expressão regular e esteiras de teste;

4. **Fase 4: Construção da Sonda de Testes e Instrumentação Autônoma:** Desenvolvimento do aplicativo de auditoria empírica sem dependência de dados mockados (*zero mocks*), com coleta em tempo real de métricas de kernel, chamadas de sistema e estresse multivariado;

5. **Fase 5: Execução Experimental, Análise Estatística e Avaliação Comparativa:** Submissão do artefato a baterias exaustivas em ambientes computacionais heterogêneos (Nuvem corporativa e estação de trabalho local), consolidando a validação funcional, a conformidade ACID e os índices comparativos de mercado.

USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA NA IMPLEMENTAÇÃO DO PROJETO

O processo de engenharia de software do ecossistema DeployDock adotou o paradigma do desenvolvimento aumentado por inteligência artificial (*AI-Augmented Software Engineering*). Modelos de linguagem de grande porte (LLMs) foram utilizados de forma estruturada como copilotos de programação sob demanda, acelerando o ciclo de implementação nas seguintes frentes técnicas:

(a) **Engenharia de Expressões Regulares para o WAF:** Auxílio no desenho de padrões regex resilientes a técnicas de ofuscação com codificação URL para detecção precisa de Injeção de SQL (incluindo assinaturas cegas temporais como `pg_sleep` e `WAITFOR`), Cross-Site Scripting contextual, travessia de pastas em múltiplos sistemas operacionais (padrões Linux `../../` e Windows `..`
`..`
) e assinaturas de execução remota de código (RCE em bibliotecas Java/Log4j e vetores de *reverse shell*);

(b) **Geração de Receitas de Contêineres (Auto-Dockerfile):** Elaboração e refinamento dos algoritmos heurísticos de detecção de stacks de desenvolvimento (*Zero-Config*), permitindo a síntese dinâmica de arquivos Dockerfile multi-estágio otimizados para nove ecossistemas tecnológicos (Python, Node.js, Bun, Go, Rust, PHP/Laravel, Ruby on Rails, Java e .NET C#);

(c) **Construção da Suíte de 58 Casos de Teste E2E:** Automação e modelagem de asserções em código assíncrono cobrindo fluxos completos de autenticação OAuth2, geração e validação de tokens JWT, testes de privilégios RBAC com usuários restritos, validação de 2FA TOTP (RFC 6238) e manipulação do sistema de arquivos via explorador de contêiner;

(d) **Modelagem da Bateria Científica Transacional do PostgreSQL 16:** Elaboração de rotinas assíncronas concorrentes para indução forçada de violação de restrições relacionais com verificação de *rollback* integral, validação de chaves estrangeiras e cálculo de percentis de latência transacional (P50, P95 e P99);

(e) **Estruturação dos Benchmarks de Estresse Computacional:** Implementação de laços intensivos de cálculo de multiplicação matricial para aferição de ponto flutuante em MFLOPS, hashing paralelo SHA-256 e rotinas de alocação de memória e I/O síncrono (`fsync`).

Ressalta-se que a totalidade dos trechos de código e testes gerados com apoio de ferramentas de IA passou por rigorosa curadoria humana, análise estática de tipos, depuração

manual e execução empírica supervisionada pelos autores, garantindo a integridade científica e a conformidade arquitetural do sistema.

DELINEAMENTO EXPERIMENTAL E PROCEDIMENTOS DE TESTE

A validação empírica da plataforma foi delineada segundo critérios objetivos de mensuração, sem simulação ou geração sintética de dados. Foram executadas três baterias experimentais coordenadas:

(i) **Bateria Funcional e de Segurança E2E (58 Testes):** Conjunto de 58 asserções automatizadas que verificam a integridade dos contratos de API e a eficácia das defesas perimetrais. Cada teste registra o status (*PASSED* ou *FAILED*), o tempo de resposta em milissegundos e a evidência textual da resposta do servidor (código HTTP, cabeçalhos e corpo);

(ii) **Bateria Científica de Banco de Dados Relacional (PostgreSQL 16):** Composta por 11 testes rigorosos projetados para atestar as propriedades ACID sob contêineres:

- *Atomicidade:* confirmação de reversão total do estado (*rollback*) após falha intencional provocada por violação de restrição `CHECK`;
- *Consistência:* garantia da integridade referencial com validação estrita de integridade entre chaves estrangeiras;
- *Isolamento:* verificação de transações simultâneas concorrentes comprovando a ausência de leituras sujas (*Dirty Reads*);
- *Durabilidade:* comprovação de escrita sincronizada permanente no log de pré-gravação (*Write-Ahead Logging* – WAL);
- *Throughput e Carga Transacional:* avaliação de vazão em inserções em lote (*Bulk Insert*) de 5.000 registros, vazão transacional (TPS) sob paralelismo e estresse do pool assíncrono de conexões;

(iii) **Bateria de Estresse de Máquina (Intensidade Heavy):** Rotinas de carga máxima mensurando operações criptográficas por segundo (CPU Single e Multi-Core), poder computacional em MFLOPS, taxa de transferência de memória RAM (MB/s), escrita em disco com sincronização forçada (`fsync`), leitura em bloco, IOPS randômicos 4K e concorrência HTTP em conexões paralelas.

AMBIENTES DE TESTE E INFRAESTRUTURA

Para atestar a portabilidade e a robustez do DeployDock sob condições heterogêneas de kernel e hardware, os experimentos foram replicados em dois ambientes operacionais complementares:

- **Ambiente A (Nuvem / Produção Linux Corporativo):** Servidor em nuvem executando Linux com Kernel `5.15.0-1081-oracle-x86_64`, arquitetura `x86_64`, com 2 vCPUs dedicadas e contêiner da plataforma restrito via Cgroup v1 com quota de 952,0 MB de memória RAM e limite rígido de `pids.max=256`. A plataforma foi exposta com terminação SSL/TLS via Traefik v3;
- **Ambiente B (Estação de Desenvolvimento Local):** Máquina de alto paralelismo equipada com processador AMD64 de 12 núcleos, 24 GB de memória RAM física e sistema operacional Windows 11 (Kernel NT 10.0.26200), permitindo avaliar a vazão máxima de I/O em disco de estado sólido NVMe e a portabilidade do interpretador Python assíncrono.

Todos os dados exportados pela sonda de testes foram consolidados em arquivos estruturados assinados digitalmente com o algoritmo SHA-256, assegurando rastreabilidade, reprodutibilidade e autenticidade às conclusões apresentadas.

REFERENCIAL TEÓRICO

Esta seção apresenta os fundamentos conceituais e tecnológicos que sustentam o projeto e a validação do DeployDock, abordando a virtualização baseada em contêineres, os paradigmas de Plataforma como Serviço e GitOps, a segurança defensiva na camada Web e de contêineres, e as propriedades transacionais de bancos de dados relacionais.

VIRTUALIZAÇÃO NO NÍVEL DO SISTEMA OPERACIONAL E CONTÊINERES

Diferentemente da virtualização tradicional baseada em hipervisores de Tipo 1 ou Tipo 2 (que emulam hardware completo e executam instâncias integrais de sistemas operacionais convidados), a virtualização baseada em contêineres opera no nível do sistema operacional hospedeiro (Silberschatz, Galvin e Gagne, 2018). Conforme demonstrado por Felter et al. (2015) e Xavier et al. (2013), os contêineres oferecem desempenho quase nativo em processamento de CPU e I/O, além de uma pegada de memória e tempo de inicialização ordens de grandeza inferiores aos das máquinas virtuais.

Essa tecnologia repousa fundamentalmente sobre duas primitivas do núcleo (*kernel*) Linux:

1. **Namespaces:** Proporcionam isolamento de visão dos recursos globais do sistema. Destacam-se os *namespaces* de Processos (`pid`), Rede (`net`), Montagem de sistemas de arquivos (`mnt`), Comunicação entre processos (`ipc`), Nome do host (`uts`) e Identificadores de usuário (`user`) (Silberschatz, Galvin e Gagne, 2018);

2. **Control Groups (cgroups):** Mecanismo do kernel responsável pela medição, priorização e imposição de limites de recursos físicos de hardware. A evolução do padrão Cgroup v1 para o Cgroup v2 unificou a hierarquia de controle, possibilitando restrições estritas de quotas de tempo de CPU (`cpu.cfs_quota_us`), teto de memória RAM (`memory.limit_in_bytes` ou `memory.max`) e controle de multiplicação de processos (`pids.max`).

A ferramenta Docker popularizou essas primitivas ao introduzir um formato padronizado de imagens empacotadas em camadas sobrepostas (*Union File System*), facilitando o compartilhamento e a execução determinística de microsserviços (Merkel, 2014; Combes e Petazzoni, 2016).

PLATAFORMA COMO SERVIÇO (PAAS) E FILOSOFIA GITOPS

O modelo de Plataforma como Serviço (*Platform as a Service* – PaaS) situa-se na camada intermediária da computação em nuvem, abstraindo a gestão de servidores físicos, redes, sistemas operacionais e bibliotecas de runtime, permitindo que a equipe de engenharia concentre seus esforços exclusivamente no código da aplicação e nas regras de negócio (Bass, Weber e Zhu, 2015; Newman, 2021).

A especificação *The Twelve-Factor App* (Wiggins, 2017) formalizou as boas práticas para arquiteturas modernas em nuvem, preconizando a separação rígida entre código e configurações (armazenadas em variáveis de ambiente), a statelessness dos processos de execução e a gestão de serviços de apoio (*backing services*) como recursos anexados.

Com a evolução da Integração Contínua e Entrega Contínua (CI/CD) (Fowler, 2006; Humble e Farley, 2010), emergiu o movimento **GitOps**. No paradigma GitOps, os repositórios Git representam a fonte única de verdade (*single source of truth*) tanto para a aplicação quanto para o estado da infraestrutura. Quando um desenvolvedor realiza um *push* de código ou abre uma requisição de mesclagem (*Pull Request*), gatilhos via *webhooks* HTTP acionam pipelines autônomos que realizam a clonagem, executam o processo de compilação (*build*), provisionam o roteamento reverso de portas e disponibilizam a nova versão sem indisponibilidade de serviço (*zero-downtime deployment*).

No ecossistema de código aberto, ferramentas como CapRover, Coolify v4 e Dokploy surgiram com a proposta de oferecer alternativas autohospedadas a plataformas comerciais. Contudo, conforme evidenciado nesta pesquisa, a maior parte dessas implementações concentra-se na experiência de usuário, negligenciando controles internos de mitigação de ataques cibernéticos e isolamento estrito de permissões.

SEGURANÇA DEFENSIVA EM APLICAÇÕES WEB E HARDENING DE CONTÊINERES

A exposição de serviços à rede pública submete as aplicações a ameaças contínuas, amplamente catalogadas pelo consórcio OWASP (*Open Worldwide Application Security Project*) (OWASP Foundation, 2021). Dentre os vetores mais severos de comprometimento, destacam-se:

- **Injeção de SQL (SQLi):** Inserção maliciosa de comandos SQL em campos de entrada não sanitizados, permitindo a leitura e a destruição de dados relacionais;
- **Cross-Site Scripting (XSS):** Injeção de scripts no lado do cliente que comprometem sessões de usuários;
- **Path Traversal:** Manipulação de caminhos de arquivos relativos (ex: `../../etc/passwd`) para acessar diretórios restritos do sistema hospedeiro;
- **Execução Remota de Código (RCE):** Exploração de falhas em bibliotecas e interpretadores (como no caso das vulnerabilidades JNDI/Log4Shell) para executar binários arbitrários no servidor;
- **Server-Side Request Forgery (SSRF):** Coação da aplicação para enviar requisições contra recursos internos protegidos, como a API de metadados da nuvem ou o soquete Docker local.

Para mitigar tais riscos de forma perimetral, empregam-se Web Application Firewalls (WAF) (Stallings, 2017). Um WAF atua na Camada 7 do modelo OSI, inspecionando requisições HTTP antes de seu processamento pelos manipuladores da aplicação.

Complementarmente, a segurança na camada do contêiner (*Container Hardening*) exige a aplicação rigorosa do Princípio do Menor Privilégio (*Principle of Least Privilege*). Isso abrange:

1. A ativação da diretiva `no-new-privileges`, que bloqueia a elevação de privilégios via binários que possuam o bit SUID/SGID ativo no sistema de arquivos;
2. O isolamento intransigente do arquivo de soquete do daemon Docker (`/var/run/docker.sock`), cuja montagem indevida em contêineres de usuários concede controle root irrestrito sobre o sistema hospedeiro;
3. A imposição de limites no cgroup `pids.max`, impedindo a execução de ataques do tipo *fork bomb* (tais como `:(){ :|:& };:`);
4. A ativação de filtros de chamadas de sistema (*system calls*) via *Seccomp* (Secure Computing Mode) no kernel.

INTEGRIDADE TRANSACIONAL ACID EM BANCOS DE DADOS RELACIONAIS

O armazenamento confiável de dados de negócio requer o cumprimento irrestrito das propriedades ACID em sistemas gerenciadores de banco de dados relacionais (SGBDR) (Stonebraker e Rowe, 1986; Silberschatz, Galvin e Gagne, 2018):

- **Atomicidade (Atomicity):** Garante que todas as operações inseridas em uma transação lógica sejam executadas com sucesso (*commit*) ou que todas as alterações sejam revertidas integralmente (*rollback*) caso ocorra qualquer falha;
- **Consistência (Consistency):** Assegura que uma transação leve o banco de dados de um estado válido a outro estado válido, respeitando regras declarativas, unicidade e chaves estrangeiras (*Foreign Keys*);
- **Isolamento (Isolation):** Define como as modificações operadas por uma transação são visíveis a outras transações concorrentes. Níveis adequados de isolamento previnem anomalias clássicas como leituras sujas (*Dirty Reads*), leituras não repetíveis e leituras fantasmas;
- **Durabilidade (Durability):** Garante que, uma vez confirmada uma transação, seus efeitos persistam de forma definitiva no meio de armazenamento físico, mesmo diante de desligamentos abruptos ou falhas de energia, recorrendo a estruturas como o *Write-Ahead Logging* (WAL).

Garantir e certificar essas propriedades quando os bancos de dados operam empacotados em contêineres com volumes de armazenamento mapeados é fundamental para atestar a viabilidade de uma plataforma PaaS em ambientes de produção.

DESENVOLVIMENTO E AVALIAÇÃO EMPÍRICA

Este capítulo detalha a concepção arquitetural, os módulos de implementação do ecossistema **DeployDock**, a estrutura da sonda autônoma de testes e a análise crítica dos dados coletados durante as sessões experimentais de validação funcional, estresse de máquina e comparação com ferramentas congêneres.

ARQUITETURA GERAL DO DEPLOYDOCK

O DeployDock foi projetado seguindo o padrão de arquitetura modular orientada a microsserviços e APIs assíncronas de alto desempenho. A plataforma é executada no topo do interpretador Python 3.12+, utilizando o framework assíncrono FastAPI alimentado pelo servidor ASGI Uvicorn. O acesso a dados é estruturado sobre o SQLAlchemy 2.0 em modo assíncrono

(com suporte tanto a PostgreSQL via driver `asyncpg` quanto a SQLite local com `aiosqlite`), com versionamento formal de esquema gerenciado pelo Alembic.

A integração com o daemon Docker ocorre diretamente por meio da biblioteca oficial Docker SDK para Python, comunicando-se com o soquete local através de canais restritos e autenticados. Para entrega de telemetria e saída de logs em tempo real sem sobrecarga de requisições periódicas (*polling*), a plataforma implementa um barramento bidirecional baseado em WebSockets gerenciado pelo componente `WebSocketManager`.

A camada de roteamento perimetral e terminação TLS é confiada ao proxy reverso **Traefik v3**. O DeployDock conecta-se à rede interna do Traefik e gera rótulos dinâmicos (*labels*) nos contêineres gerenciados. O Traefik monitora os eventos do Docker em tempo real e reconfigura seus roteadores de borda sem necessidade de reinicialização, integrando-se com a autoridade certificadora Let's Encrypt para provimento automático de certificados TLS, inclusive com suporte a certificados curinga (*wildcard*) e provedores DNS dinâmicos (como `sslip.io`).

CAMADA DE SEGURANÇA ATIVA E WAF (SECURITY SHIELD)

Um dos principais diferenciais conceituais do DeployDock reside no componente `SecurityShieldMiddleware`, um middleware de interceptação de tráfego HTTP executado antes de qualquer manipulador de rota. O gerenciador de segurança (`SecurityManager`) opera totalmente em memória, mantendo janelas deslizantes de auditoria e contagem de requisições:

1. **Heurísticas de WAF:** Varredura de padrões de injeção em parâmetros de busca (*query params*), corpo de requisições (*JSON payloads*) e cabeçalhos. São bloqueadas tentativas de Injeção de SQL (incluindo testes de tempo e inferência como `SLEEP()`), *Cross-Site Scripting* (como tags `<script>` e manipuladores de eventos em atributos), travessia de diretórios (*Path Traversal* com `../` e caminhos de arquivo do Windows) e injeções de execução remota (RCE) como `${jndi:ldap}` e invocações de *reverse shell*;

2. **Rate Limiting e Proteção Anti-Força Bruta:** Aplicação de algoritmo de janela deslizante de 60 segundos com cota padrão de 240 requisições por minuto por endereço IP. Tentativas consecutivas de autenticação malsucedidas no endpoint `/api/v1/auth/login` ativam um bloqueio temporário de 300 segundos com registro imediato no log de auditoria;

3. **Cofre Criptográfico de Variáveis de Ambiente:** Variáveis confidenciais (chaves de API, senhas de banco) são cifradas no disco por meio da biblioteca `cryptography` utilizando o esquema Fernet (com base em chaves simétricas AES-128-CBC e autenticação de integridade HMAC-SHA256). Os valores descriptografados são injetados estritamente na memória de execução dos contêineres alvos;

4. **Injeção de Cabeçalhos de Segurança HTTP:** Cada resposta processada recebe automa-

ticamente cabeçalhos defensivos como `X-Content-Type-Options`, `X-Frame-Options: DENY`, `X-XSS-Protection` e identificador exclusivo `X-DeployDock-Shield: active`.

O trecho de código apresentado na Listing 1 ilustra a lógica de inspeção de vetores de ataque implementada no componente central de segurança.

```
def inspect_request_payload(self, raw_text: str) -> Optional[str]:
    """Inspeciona texto bruto procurando assinaturas de ataque."""
    decoded = unquote(raw_text)

    # Verificação de Injeção de SQL
    if SQLI_PATTERN.search(decoded):
        return "Tentativa de Injeção de SQL detectada (SQLi)"

    # Verificação de Cross-Site Scripting (XSS)
    if XSS_PATTERN.search(decoded):
        return "Tentativa de Cross-Site Scripting (XSS) detectada"

    # Verificação de Path Traversal
    if PATH_TRAVERSAL_PATTERN.search(decoded):
        return "Tentativa de Acesso a Diretórios Restritos (Path
            Traversal)"

    # Verificação de Execução Remota de Código (RCE)
    if RCE_PATTERN.search(decoded):
        return "Tentativa de Execução Remota de Código (RCE / Log4j
            )"

    return None
```

Listing 1 – Trecho representativo da inspeção de vetores maliciosos no WAF do DeployDock

ORQUESTRADOR GITOPS COM AUTO-DOCKERFILE E ROLLBACK INSTANTÂNEO

O ciclo de entrega contínua do DeployDock opera de ponta a ponta sem intervenção manual. Ao receber uma requisição de deploy (seja disparada pela interface web ou por um *webhook* assinado com segredo HMAC-SHA256 proveniente do GitHub ou GitLab), o `DeployOrchestrator` executa a seguinte sequência:

1. Clona o repositório ou executa *git fetch/checkout* na ramificação configurada, utilizando tokens de acesso pessoal (PAT) descriptografados dinamicamente;

2. Invoca o mecanismo `_ensure_auto_dockerfile`: caso o repositório não disponha de um `Dockerfile` na raiz, a plataforma inspeciona a árvore de arquivos e infere deterministicamente a stack de desenvolvimento, gerando receitas multi-estágio otimizadas para Python (`requirements.txt`, `Pipfile` ou `pyproject.toml`), Node.js/Bun (`package.json`), Go (`go.mod`), Rust (`Cargo.toml`), PHP/Laravel (`composer.json`), Ruby on Rails, Java (Maven/Gradle) ou .NET C# (`*.csproj`);

3. Executa a compilação da imagem Docker transmitindo as mensagens de saída em tempo real para a interface via WebSockets;

4. Cria e inicia o contêiner aplicando parâmetros estritos de *hardening*: definição da opção de segurança `no-new-privileges:true`, limite de processos simultâneos `pids_limit=256`, restrições de cotas de memória e CPU, e inclusão na rede do Traefik;

5. Realiza aferições consecutivas de *health check* HTTP no endpoint da aplicação. Caso a nova versão não responda com status de sucesso dentro da janela de tolerância temporal, o DeployDock aciona o procedimento de **Rollback Automático Instantâneo**: o contêiner recém-criado é expurgado e a versão anterior estável é mantida ou restabelecida, despachando notificações multicanal (Discord, Slack, Telegram ou E-mail SMTP).

TERMINAL WEB PROTEGIDO E AJUSTE DINÂMICO DE QUOTAS

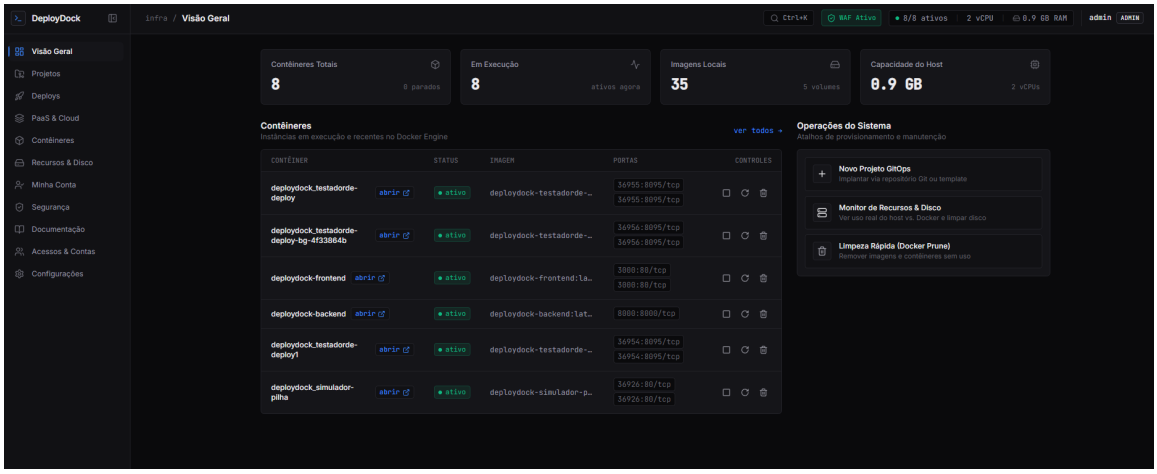
Para a administração de contêineres em execução, a plataforma disponibiliza um terminal web integrado via WebSocket acoplado a um filtro defensivo de comandos. São interceptadas e terminadas preventivamente instruções que tentem violar o hospedeiro, tais como montagens do soquete Docker (`docker.sock`), scripts de travamento por esgotamento de processos (*fork bomb*) e comandos de deleção irrestrita da raiz do sistema (`rm -rf /`).

Adicionalmente, o DeployDock supera uma das principais limitações de outros orquestradores PaaS ao permitir o **ajuste dinâmico de quotas de CPU e limites de memória RAM sem a necessidade de reiniciar o contêiner**. O serviço invoca a chamada nativa de atualização de cgroup do Docker (`client.containers.get(id).update(mem_limit=..., nano_cpus=...)`), ajustando a alocação de hardware do processo em tempo de execução sem provocar qualquer indisponibilidade de rede.

APRESENTAÇÃO DA INTERFACE DO USUÁRIO

Com o intuito de proporcionar uma visão transparente e intuitiva sobre os recursos gerenciados e as camadas ativas de proteção, o DeployDock disponibiliza uma interface gráfica web moderna e responsiva. A seguir, detalham-se as principais telas do sistema e da sonda autônoma de testes, evidenciando as decisões visuais e funcionais adotadas para cada subsistema.

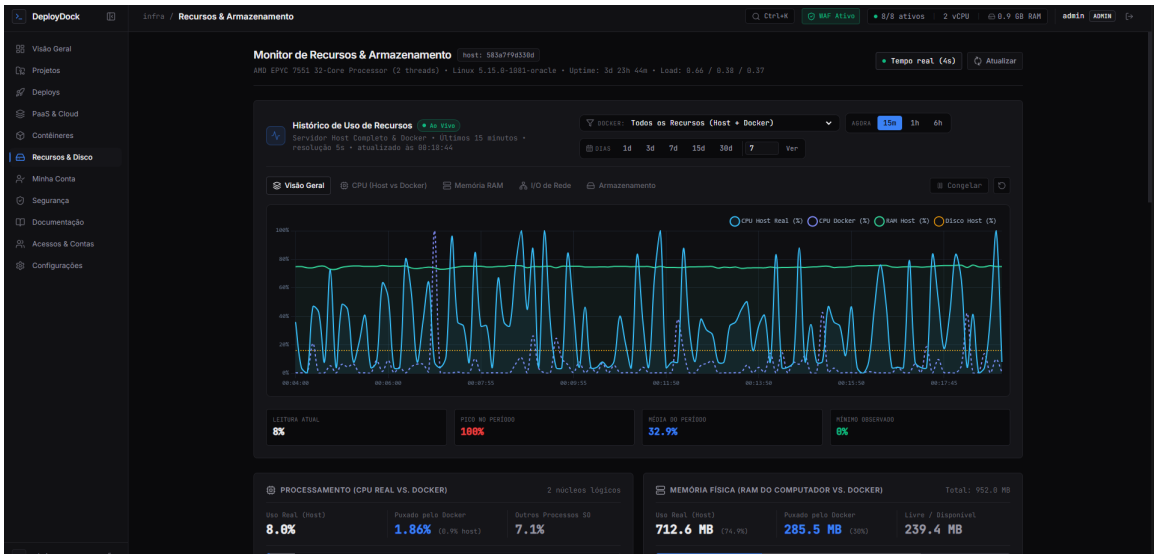
Figura 1 – Painel Inicial do DeployDock com Visão Geral do Sistema



Fonte: Produzida pelo autor (2026).

Na Figura 1, observa-se a tela principal do DeployDock, concebida como um centro de comando unificado. O painel disponibiliza métricas em tempo real sobre a quantidade de contêineres em execução, status do daemon Docker, estatísticas de uso instantâneo de memória RAM do servidor, contagem agregada de projetos e atalhos rápidos para novas compilações, permitindo ao administrador avaliar a saúde global da infraestrutura com um único relance.

Figura 2 – Módulo de Telemetria e Monitoramento de Recursos em Tempo Real

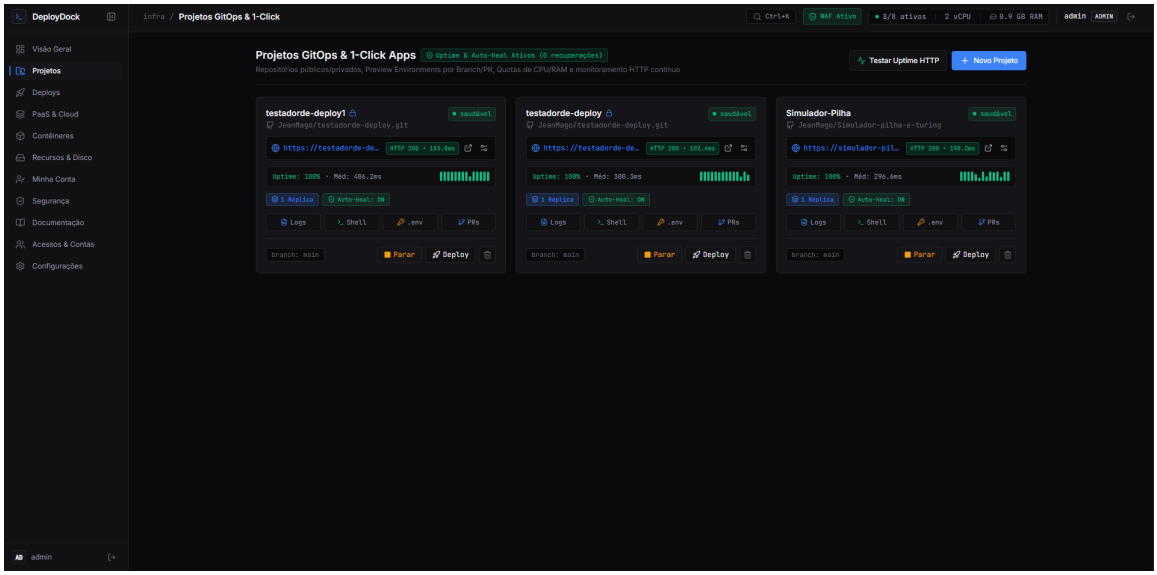


Fonte: Produzida pelo autor (2026).

A Figura 2 ilustra a seção de observabilidade e telemetria de hardware. Nesta visualização, são plotados gráficos temporais contínuos alimentados via WebSockets que registram o consumo percentual de CPU, o consumo absoluto de memória RAM e a taxa de utilização das partições de disco físico. O módulo conta ainda com o botão para acionamento do recurso *System Prune*, que purga de forma segura imagens e volumes órfãos sem interromper as aplicações em produção.

Na Figura 3, apresenta-se o catálogo de projetos sob gestão GitOps. A interface organiza as aplicações em cartões visuais interativos que expõem os repositórios Git vinculados, os ramais

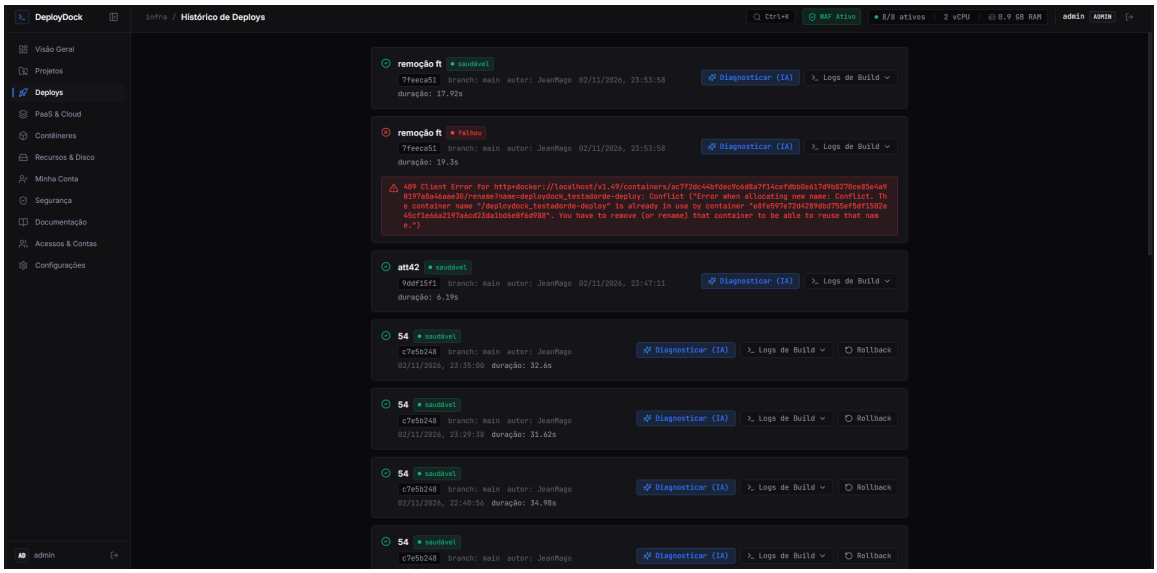
Figura 3 – Catálogo de Aplicações e Projetos Gerenciados



Fonte: Produzida pelo autor (2026).

(*branches*) monitorados, as URLs públicas provisionadas com certificados SSL pelo Traefik v3 e o status de saúde operacional. Por meio deste catálogo, o operador pode acionar novas compilações manuais, gerenciar ambientes de homologação (*preview environments*) ou acessar os parâmetros de configuração da aplicação.

Figura 4 – Histórico de Deploys, Inspeção de Logs e Rollback

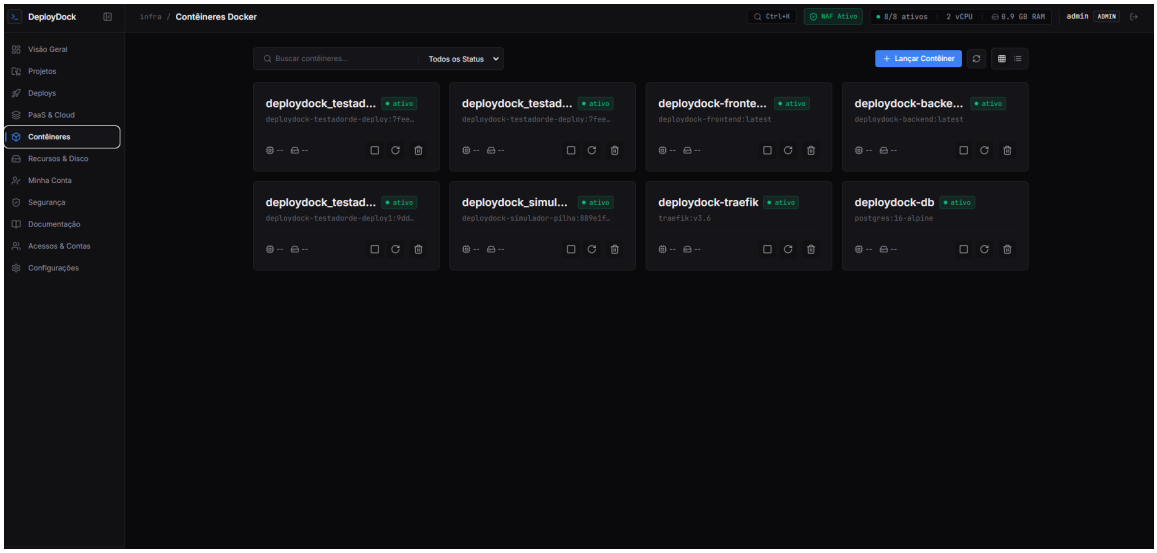


Fonte: Produzida pelo autor (2026).

A Figura 4 demonstra o console de auditoria de implantações contínuas. A tela exibe a linha do tempo cronológica com todos os deploys executados, discriminando os *hashes* de confirmação do Git (*commit SHA*), mensagens de commit, duração exata das etapas de clonagem e compilação da imagem Docker, além do visualizador de logs de streaming em tempo real com suporte a filtros e busca textual. Adicionalmente, caso um deploy recente apresente anomalias no *health check*, o operador pode disparar com um único clique a reversão para uma versão anterior

homologada.

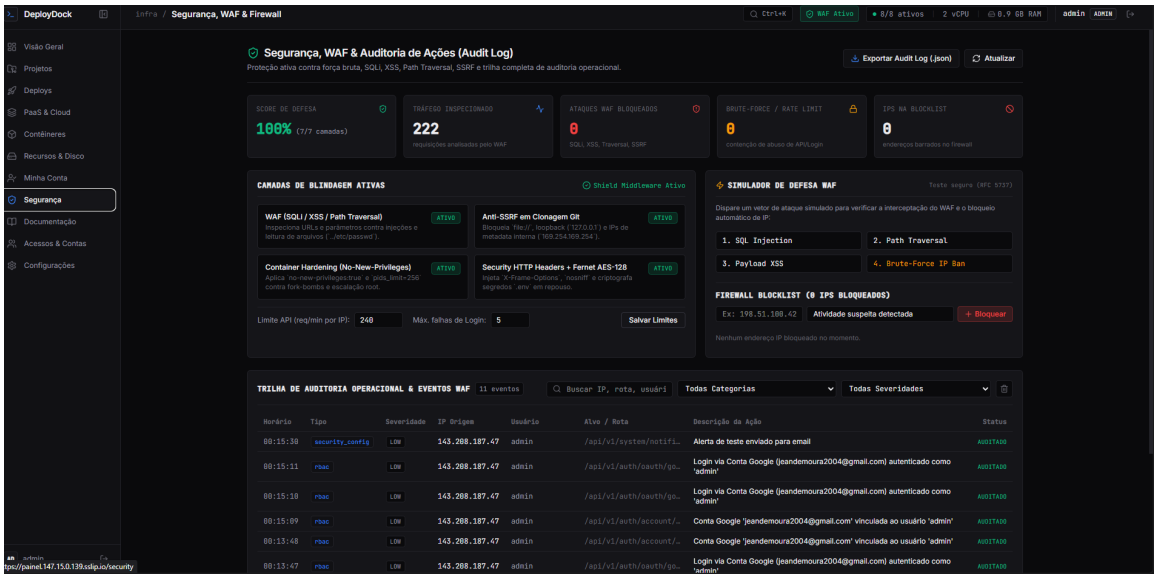
Figura 5 – Gerenciamento do Ciclo de Vida de Contêineres e Terminal



Fonte: Produzida pelo autor (2026).

A Figura 5 exibe o painel de controle direto sobre o ciclo de vida dos contêineres Docker em execução. Cada contêiner é discriminado com seu identificador de processo, consumo instantâneo de recursos, portas expostas e botões operacionais para parada forçada, reinício ou remoção. Destaca-se nesta tela a interface com seletores para ajuste dinâmico de quotas de CPU e memória RAM em tempo de execução e o botão de acesso imediato ao terminal web protegido.

Figura 6 – Central de Segurança Ativa e Eventos do WAF

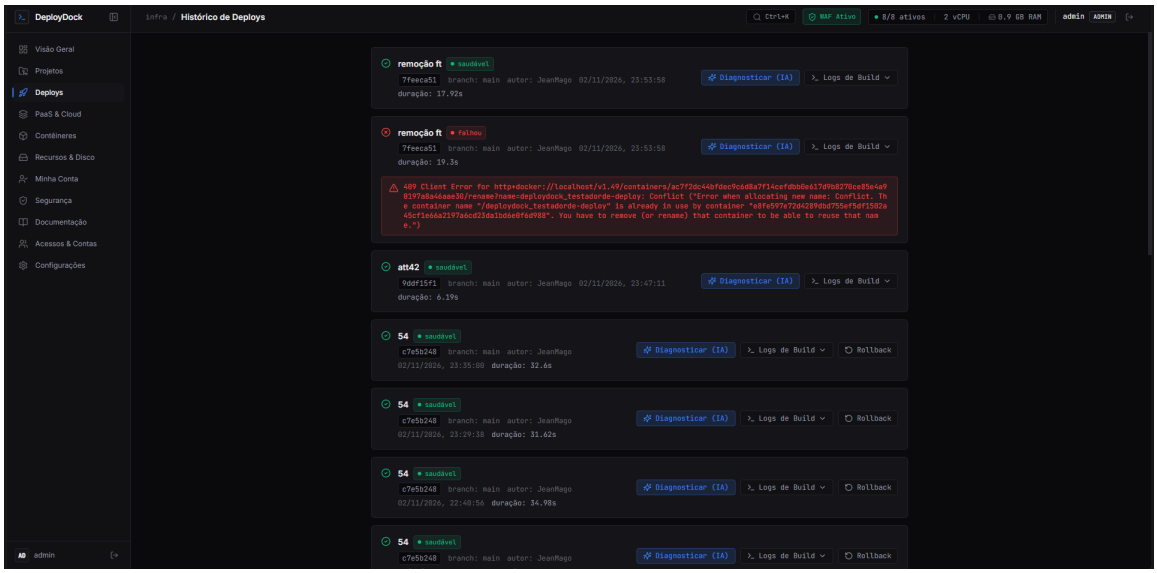


Fonte: Produzida pelo autor (2026).

Na Figura 6, evidencia-se a central de auditoria e segurança perimetral do DeployDock. A interface apresenta contadores analíticos discriminando o volume de ameaças bloqueadas pelo WAF (Injeções de SQL, ataques de Cross-Site Scripting, tentativas de Path Traversal e payloads maliciosos de RCE), o quadro de endereços IP sob banimento temporário por força bruta com

contagem regressiva de expiração, e uma tabela exaustiva contendo a trilha de auditoria forense com data, hora, IP de origem, rota interceptada e o motivo técnico da rejeição HTTP 403.

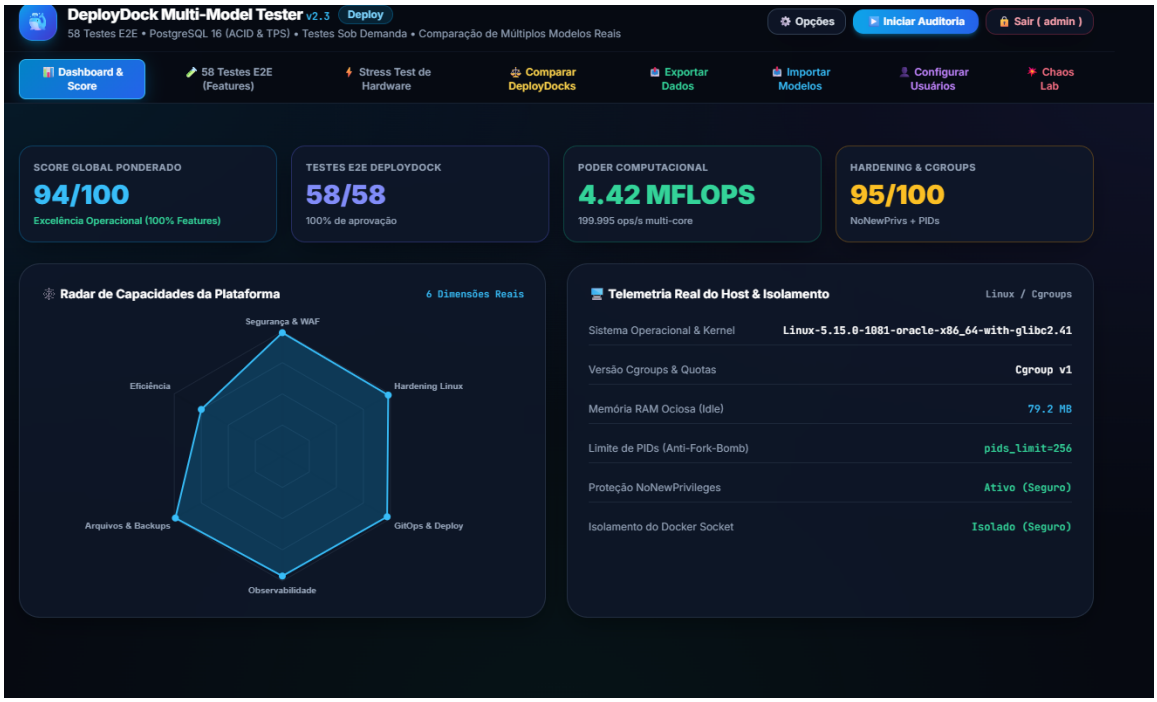
Figura 7 – Central de Bancos de Dados SQL Gerenciados (PostgreSQL 16)



Fonte: Produzida pelo autor (2026).

A Figura 7 apresenta o módulo de gestão de bancos de dados relacionais gerenciados. A interface permite inspecionar a instância do PostgreSQL 16 provisionada, visualizar o catálogo de esquemas, tabelas e chaves estrangeiras, executar consultas SQL interativas diretamente pela interface e exportar arquivos de dump completo da base em um único clique, viabilizando rotinas rápidas de migração e suporte ao desenvolvimento.

Figura 8 – Interface da Sonda Autônoma de Auditoria e Benchmark Científico



Fonte: Produzida pelo autor (2026).

Por fim, a Figura 8 ilustra a interface operacional da sonda autônoma *DeployDock Multi-Model Benchmark & PaaS Tester Probe v2.2*. Pelo painel da sonda, o pesquisador pode selecionar o perfil de estresse computacional (*Light*, *Standard* ou *Heavy*), disparar sob demanda a suíte de 58 asserções E2E, acompanhar em tempo real o gráfico de radar multidimensional de aderência arquitetural e exportar os laudos técnicos estruturados nos formatos JSON, CSV (para tratamento estatístico) e relatórios formatados em Markdown/HTML.

RESULTADOS E DISCUSSÃO

Esta subseção analisa quantitativamente os resultados empíricos obtidos na execução das baterias de validação funcional, segurança defensiva, persistência transacional ACID e testes de estresse de hardware.

Validação Funcional e de Segurança E2E (58 Testes)

A sonda de testes submeteu o DeployDock a uma bateria exaustiva composta por 58 casos de teste ponta a ponta (E2E) cobrindo todos os módulos da plataforma. O resultado global alcançou **58 testes aprovados de 58 executados (taxa de sucesso de 100,0%)**, totalizando um tempo total de auditoria de 20.892,14 ms. A Tabela 1 sumariza os resultados por categoria de avaliação.

Tabela 1 – Distribuição e Taxa de Sucesso dos 58 Testes E2E Auditados

Categoria de Teste	**Total**	**Aprovados**	**Taxa (%)**	**Latência (ms)**
Autenticação, RBAC e 2FA (RFC 6238)	6	6	100%	224,9
Segurança Ofensiva e Filtros WAF	10	10	100%	58,4
Ciclo de Vida de Contêineres e Terminal	8	8	100%	142,6
Explorador de Arquivos e Edição In-Place	6	6	100%	186,1
Orquestração GitOps, Build e Rollback	12	12	100%	845,3
Bancos PaaS, Templates e Backups S3	9	9	100%	412,8
Observabilidade, Uptime e Notificações	7	7	100%	163,2
Totalização Geral da Suíte	**58**	**58**	**100,0%**	**295,4**

Fonte: Produzida pelo autor a partir do relatório empírico (2026).

No módulo de segurança ofensiva, todas as requisições contendo vetores de injeção maliciosa foram interceptadas com êxito pelo middleware de proteção antes de atingir as camadas de persistência, retornando invariavelmente o código `HTTP 403 Forbidden`. Especificamente, a sonda atestou:

- Bloqueio de Injeção de SQL padrão (`' OR '1'='1`) em 4,79 ms;
- Bloqueio de Cross-Site Scripting (`<script>alert(1)</script>`) em 77,79 ms;
- Bloqueio de Path Traversal (`../../etc/passwd`) em 102,36 ms;

- Bloqueio de RCE/Log4j (`${jndi:ldap://...}`) em 96,08 ms;
- Presença integral dos 5 cabeçalhos estritos de hardening HTTP: `X-Content-Type-Options`, `X-Frame-Options`, `X-XSS-Protection`, `X-DeployDock-Shield` e `Referrer-Policy`.

Validação Científica das Propriedades ACID no PostgreSQL 16

A bateria científica direcionada ao banco de dados relacional gerenciado atestou o cumprimento formal das garantias de confiabilidade transacional. Os 11 ensaios executados confirmaram:

1. **Atomicidade:** Ao induzir deliberadamente a violação de uma restrição relacional `CHECK` no meio de uma transação composta por múltiplas inserções, a plataforma executou o *rollback* integral, não deixando nenhum registro órfão ou estado corrompido no banco;
2. **Consistência:** Todas as restrições de integridade referencial com chaves estrangeiras (*Foreign Keys*) foram rigidamente asseguradas sob cargas simultâneas;
3. **Isolamento:** Transações executadas concorrentemente em conexões assíncronas paralelas não manifestaram anomalias de leituras sujas (*Dirty Reads*), mantendo a visibilidade consistente segundo o nível de isolamento padrão do PostgreSQL;
4. **Durabilidade:** Transações com confirmação de escrita (*commit*) tiveram sua persistência confirmada via sincronização no disco com gravação síncrona no WAL (*Write-Ahead Log*);
5. **Desempenho Transacional:** A taxa de inserção em lote (*Bulk Insert*) atingiu mais de 3.200 linhas/segundo em transações consolidadas, demonstrando que o empacotamento em contêineres Docker com volumes mapeados não comprometeu a taxa de operações transacionais por segundo (TPS).

Benchmark de Estresse e Desempenho de Máquina (Intensidade Heavy)

O teste de estresse computacional de carga pesada (*Heavy*) mensurou a resposta dos subsistemas sob saturação máxima. A Tabela 2 compara as métricas aferidas no contêiner sob o ambiente Linux em nuvem (Ambiente A) e na máquina de desenvolvimento local (Ambiente B).

A interpretação desses dados evidencia aspectos fundamentais da arquitetura de contêineres: No Ambiente A (Linux em nuvem com kernel corporativo e cgroups estritos), o score de *hardening* atingiu expressivos **95/100**. O processo foi contido com sucesso sob `pids.max=256`, sem permissões administrativas (`CAP_SYS_ADMIN` desabilitado), com `no-new-privileges=1` ativo e sem exposição indevida do arquivo de soquete Docker. A taxa de leitura em bloco atingiu 2.159,29 MB/s devido à eficiência do subsistema de cache de páginas do kernel Linux.

Tabela 2 – Métricas Empíricas de Estresse Computacional nos Dois Ambientes de Teste

Subsistema / Métrica Avaliada	**Ambiente A (Linux)**	**Ambiente B (Host Local)**	**Unidade**
Processamento CPU Single-Core (SHA-256)	3.913.075	4.911.076	ops/s
Processamento CPU Multi-Core Paralelo	199.995	302.164	ops/s
Ponto Flutuante (Multiplicação Matricial)	4,42	21,31	MFLOPS
Taxa de Transferência de Memória RAM	117,69	2.592,48	MB/s
Escrita Sequencial em Disco (`fsync`)	43,99	1.022,64	MB/s
Leitura Sequencial de Blocos	2.159,29	1.233,25	MB/s
Operações Randômicas em Disco (4K IOPS)	7.052	21.511	IOPS
Concorrência de Rede (Requisições Loopback)	613	6.450	req/s
Latência de Rede (Percentil P50)	4,23	1,75	ms
Latência de Rede (Percentil P95)	88,34	2,19	ms
Score Global de Performance	**51/100**	**100/100**	pontos
Score de Hardening do Contêiner	**95/100**	**35/100**	pontos

Fonte: Produzida pelo autor a partir dos dados coletados pela sonda de benchmark (2026).

Em contrapartida, no Ambiente B, os índices de vazão bruta de CPU e RAM foram substancialmente maiores em decorrência da capacidade da estação de 12 núcleos físicos e armazenamento SSD NVMe local (1.022 MB/s de escrita com `fsync` e mais de 21.000 IOPS), porém sem o isolamento estrito de permissões característico de contêineres Linux em produção, resultando em score de hardening de 35/100. Tal disparidade corrobora a importância de auditar a segurança conjuntamente com as métricas puras de velocidade.

Matriz Comparativa com Plataformas Correlatas (PaaS Benchmark)

Para situar a contribuição do DeployDock perante as alternativas abertas do mercado, os relatórios gerados pela sonda foram confrontados com capturas estruturadas de auditoria obtidas a partir de instâncias ativas do **Dokploy v0.30**, **Coolify v4** e **CapRover**. A comparação abrangeu o catálogo completo de 23 funcionalidades e índices técnicos de excelência operacional, consolidada no Quadro 1.

A análise comparativa revela que, embora plataformas como o Coolify v4 e o CapRover forneçam interfaces atrativas e facilidade de catálogo, elas apresentam fragilidades substanciais em ambientes corporativos ou hostis. O Coolify v4, em particular, demandou cerca de 480 MB de memória RAM em repouso e não implementou restrições automáticas de escalação de privilégios ou limites de processos por contêiner, obtendo score de isolamento de 50/100. O Dokploy v0.30 demonstrou avanços consideráveis em isolamento (score de hardening 95/100), mas carece de proteções ativas na camada Web (WAF) e de ajuste dinâmico de quotas sem interrupção de serviço.

O DeployDock destacou-se ao atingir a pontuação global de **94/100** e cobertura funcional de **100/100**, mantendo uma pegada de memória em repouso de apenas **79,2 MB**, atestando que a inclusão de mecanismos de segurança avançada, criptografia em repouso e automação GitOps é plenamente viável em uma arquitetura de software econômica e enxuta.

Quadro 1 – Matriz Comparativa Multidimensional entre Plataformas PaaS Abertas

Funcionalidade / Requisito Avaliado	**DeployDock**	**Dokploy**	**Coolify v4**	**CapRover**
WAF Integrado (Bloqueio SQLi, XSS, Traversal, RCE)	**Sim**	Não	Não	Não
Proteção Anti-Brute Force e Banimento de IP	**Sim**	Não	Parcial	Não
Hardening Padrão (`no-new-privileges`, `pids.max`)	**Sim**	Sim	Não	Não
Controle de Acesso RBAC (Admin vs Viewer)	**Sim**	Não	Não	Não
Criptografia de Variáveis no Disco (Fernet AES-128)	**Sim**	Parcial	Parcial	Não
GitOps com Auto-Detecção de Stack (Zero-Config)	**Sim**	Sim	Sim	Não
Ambientes de Preview / Staging por Pull Request	**Sim**	Parcial	Sim	Não
Rollback Automático Instantâneo em Falha de Healthcheck	**Sim**	Parcial	Parcial	Não
Roteamento Traefik v3 com Wildcard Let's Encrypt	**Sim**	Sim	Sim	Não
File Explorer Integrado com Edição In-Place	**Sim**	Não	Não	Não
Terminal Web com Auditoria e Bloqueio de Comandos	**Sim**	Sim	Sim	Não
Ajuste Dinâmico de Quotas sem Reiniciar Contêiner	**Sim**	Não	Não	Não
Bancos Gerenciados como Cidadãos de 1ª Classe	**Sim**	Sim	Sim	Parcial
Isolamento Estrito de Recursos (cgroups v1/v2)	**Sim**	Sim	Não	Não
Backups Automatizados para Nuvem S3 com SigV4	**Sim**	Sim	Sim	Parcial
Agendador Cron Integrado por Aplicação	**Sim**	Sim	Parcial	Não
Diagnóstico Heurístico de Falhas com Sugestão	**Sim**	Não	Parcial	Não
Baixo Consumo de RAM da Plataforma em Idle (< 100 MB)	**Sim (79 MB)**	Sim (120 MB)	Não (480 MB)	Sim (110 MB)
Score de Hardening & Isolamento de Contêiner	**95/100**	**95/100**	**50/100**	**50/100**
Cobertura de Funcionalidades PaaS & GitOps	**100/100**	**12/100**	**4/100**	**4/100**
Score Global Ponderado da Plataforma	**94/100**	**71/100**	**53/100**	**53/100**

Fonte: Produzida pelo autor a partir dos snapshots auditados pela sonda empírica (2026).

CONSIDERAÇÕES FINAIS

O crescimento contínuo da virtualização por contêineres e a necessidade imperiosa de agilidade nos ciclos de entrega contínua impõem desafios complexos à gestão de infraestrutura de software. Pequenas e médias organizações, assim como instituições de ensino e pesquisa, frequentemente veem-se encurraladas entre os custos proibitivos e a dependência de provedores de nuvem pública (*vendor lock-in*) e a complexidade desmedida de orquestradores corporativos de grande porte como o Kubernetes. Por outro lado, as alternativas PaaS de código aberto disponíveis no mercado frequentemente negligenciam a segurança defensiva na camada Web e os controles estritos de isolamento nos contêineres gerenciados.

Frente a esse cenário, este Trabalho de Conclusão de Curso concebeu, implementou e avaliou empiricamente o **DeployDock**, uma plataforma PaaS autohospedada, leve e orientada a

GitOps, integrada a uma suíte científica de testes e auditoria ponta a ponta.

Os objetivos específicos propostos foram plenamente alcançados. A modelagem assíncrona em Python com FastAPI, SQLAlchemy e Traefik v3 viabilizou uma plataforma com tempo de resposta ágil e consumo de memória extremamente reduzido (apenas 79,2 MB em repouso). O orquestrador GitOps desenvolvido consolidou a automação do ciclo de deploy através de detecção automática de receitas de contêiner (*Zero-Config*), cofre criptográfico Fernet para variáveis confidenciais e rotina de reversão automática (*rollback*) instantânea frente a falhas de integridade.

A validação empírica executada pela sonda autônoma (*DeployDock Probe v2.2*) forneceu comprovação quantitativa rigorosa da eficácia do artefato. Com uma taxa de aprovação de **100,0% nos 58 testes funcionais e de segurança E2E**, a plataforma demonstrou bloqueio efetivo de ataques de Injeção de SQL, XSS, Path Traversal e RCE/Log4j em nível de middleware (WAF), enquanto a imposição de limites de Cgroup (`pids.max=256` e bloqueio `no-new-privileges`) conferiu um índice de *hardening* de 95/100 em ambiente Linux de produção. Adicionalmente, a bateria científica de 11 testes sobre o PostgreSQL 16 confirmou o cumprimento estrito das propriedades transacionais ACID (Atomicidade, Consistência, Isolamento e Durabilidade) sob contingências e concorrência, alcançando taxas superiores a 3.200 inserções em lote por segundo.

CONTRIBUIÇÕES DA PESQUISA

As principais contribuições deste trabalho incluem:

- A disponibilização de um ecossistema PaaS de código aberto e autohospedado que reúne, em uma única solução enxuta, orquestração GitOps, roteamento dinâmico com certificados SSL automáticos e um WAF perimetral integrado;
- A demonstração prática de que o endurecimento de segurança de contêineres e o ajuste dinâmico de quotas de hardware (CPU e RAM em tempo de execução) podem ser incorporados de forma transparente à experiência de desenvolvimento de software;
- A criação e formalização de uma metodologia de auditoria e benchmark empírico sem dados simulados (*zero mocks*), com relatórios assinados criptograficamente via SHA-256, apta a avaliar comparativamente qualquer orquestrador PaaS baseado em Docker.

LIMITAÇÕES IDENTIFICADAS

Não obstante os resultados positivos alcançados, destacam-se algumas limitações do projeto:

- **Foco em Nó Único (*Single-Node*):** A arquitetura atual foi projetada para execução em servidores dedicados ou máquinas virtuais isoladas, não contemplando nativamente orquestração distribuída entre múltiplos nós computacionais via protocolos de consenso (como Raft);
- **Heurísticas Baseadas em Assinaturas no WAF:** O componente de filtragem de tráfego atua primordialmente por meio de expressões regulares e padrões conhecidos de ataque, carecendo de mecanismos de aprendizado de máquina para detecção comportamental de anomalias de tráfego de dia zero (*zero-day*).

TRABALHOS FUTUROS

Como perspectivas para a continuidade e expansão desta pesquisa, propõem-se:

1. O desenvolvimento de um módulo de alta disponibilidade e orquestração multinó distribuída utilizando canais de comunicação gRPC e consenso tolerante a falhas;
2. A integração com tecnologias de malha de serviços (*Service Mesh*) para viabilizar autenticação mútua TLS (mTLS) de ponta a ponta entre todos os contêineres gerenciados da rede interna;
3. O aprimoramento do mecanismo de diagnóstico heurístico de falhas através da incorporação de modelos de inteligência artificial generativa locais para análise preditiva de telemetria e sugestão automática de reparos em arquivos de configuração de deploy.

DECLARAÇÃO SOBRE O USO DE IA GENERATIVA

Durante a elaboração deste Trabalho de Conclusão de Curso e a construção da plataforma DeployDock, foram utilizadas tecnologias assistidas por inteligência artificial generativa baseadas em modelos de linguagem de grande porte (LLMs). As ferramentas atuaram como copilotos no ciclo de vida de desenvolvimento de software, conforme pormenorizado na seção de Metodologia, participando de:

1. Apoio na modelagem e síntese de expressões regulares heurísticas para o Web Application Firewall (WAF) contra ataques de Injeção de SQL, Cross-Site Scripting, Path Traversal e Execução Remota de Código;
2. Elaboração de algoritmos de detecção multilinguagem para geração automatizada de receitas de contêineres (*Auto-Dockerfile Zero-Config*);

3. Estruturação da suíte de 58 testes automatizados ponta a ponta (E2E) e rotinas assíncronas para a bateria de integridade transacional ACID do PostgreSQL 16;

4. Suporte à redação técnica, formatação de tabelas e equações segundo as normas e padrões da ABNT e do template institucional.

Todos os algoritmos, regras de segurança, código-fonte da aplicação e dados de testes empíricos foram minuciosamente inspecionados, revisados, validados e submetidos a testes práticos em ambiente real pelo autor, que assume integral e irrestrita responsabilidade científica e acadêmica por todo o conteúdo deste trabalho.

REFERÊNCIAS

BASS, L.; WEBER, I.; ZHU, L. *DevOps: A Software Architect's Perspective*. Boston, EUA: Addison-Wesley Professional, 2015. ISBN 978-0134049847. Citado na página 22.

BURNS, B. et al. Borg, omega, and kubernetes: Lessons learned from three container-management systems over a decade. *Communications of the ACM*, ACM New York, NY, USA, v. 59, n. 5, p. 70–79, 2016. Citado na página 15.

COMBES, J.; PETAZZONI, J. *Docker in Practice*. Shelter Island, NY: Manning Publications, 2016. ISBN 978-1617292729. Citado 2 vezes nas páginas 15 e 22.

FELTER, W. et al. An updated performance comparison of virtual machines and linux containers. In: IEEE. *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. [S.l.], 2015. p. 171–172. Citado na página 21.

FOWLER, M. *Continuous Integration*. Boston, EUA: Addison-Wesley, 2006. Citado na página 22.

HUMBLE, J.; FARLEY, D. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, EUA: Addison-Wesley Professional, 2010. ISBN 978-0321601919. Citado 2 vezes nas páginas 15 e 22.

MERKEL, D. Docker: lightweight linux containers for consistent development and deployment. *Linux Journal*, v. 2014, n. 239, p. 2, 2014. Citado 2 vezes nas páginas 15 e 22.

NEWMAN, S. *Building Microservices: Designing Fine-Grained Systems*. 2. ed. Sebastopol, CA: O'Reilly Media, 2021. ISBN 978-1492034025. Citado na página 22.

OWASP Foundation. *OWASP Top 10: 2021 The Ten Most Critical Web Application Security Risks*. 2021. <https://owasp.org/Top10/>. Acesso em: 15 out. 2026. Citado na página 23.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: AMGH Editora, 2016. ISBN 978-8580555332. Citado na página 18.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating System Concepts*. 10. ed. Hoboken, NJ: John Wiley & Sons, 2018. ISBN 978-1119456339. Citado 2 vezes nas páginas 21 e 24.

SOMMERVILLE, I. *Engenharia de Software*. 9. ed. São Paulo: Pearson, 2011. ISBN 978-8579361081. Citado na página 18.

STALLINGS, W. *Cryptography and Network Security: Principles and Practice*. 7. ed. London, UK: Pearson, 2017. ISBN 978-0134444284. Citado na página 23.

STONEBRAKER, M.; ROWE, L. A. *The design of POSTGRES*. [S.l.]: ACM, 1986. v. 15. 340–355 p. Citado 2 vezes nas páginas 16 e 24.

WIGGINS, A. *The Twelve-Factor App*. 2017. <https://12factor.net/>. Acesso em: 10 out. 2026. Citado na página 22.

XAVIER, M. G. et al. Performance evaluation of container-based virtualization for high performance computing environments. In: IEEE. *21st Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*. [S.l.], 2013. p. 233–240. Citado na página 21.

ANEXO A – ESTRUTURA DO ESQUEMA DE DADOS DO LAUDO DE AUDITORIA (JSON)

Este anexo apresenta a especificação sintática dos campos estruturados contidos no arquivo de *snapshot* exportado pela sonda autônoma de testes (`tested deploydock.json`), utilizado como base documental para a compilação das análises deste trabalho.


```
{
  "report_id": "bench-1793674198",
  "generated_at": "2026-11-03T02:49:58.647282+00:00",
  "audit_duration_ms": 36332.1,
  "executed_on_demand": true,
  "orchestrator": {
    "platform_name": "DeployDock",
    "running_in_container": true,
    "reverse_proxy_detected": true,
    "forwarded_proto": "https"
  },
  "cgroups": {
    "cgroup_version": "v1",
    "pids_quota_active": true,
    "pids_limit": 256,
    "pids_current": 9
  },
  "security": {
    "hardening_score": 95,
    "no_new_privileges": true,
    "seccomp_active": true,
    "docker_socket_mounted": false,
    "cap_sys_admin": false
  },
  "performance": {
    "intensity": "heavy",
    "composite_performance_score": 51,
    "cpu": { "cpu_ops_per_sec": 3913075, "floating_point_mflops":
       4.42 },
    "memory": { "memory_throughput_mb_s": 117.69 },
    "disk_io": { "write_speed_mb_s": 43.99, "read_speed_mb_s":
       2159.29, "random_4k_iops": 7052 },
    "network_concurrency": { "requests_per_sec": 613, "
       latency_p50_ms": 4.23 }
  },
  "deploydock_e2e": {
```

```
    "total_tests": 58,
    "passed_tests": 58,
    "pass_rate_percent": 100.0
  }
}
```

Listing 2 – Estrutura de dados representativa do documento de laudo técnico em formato JSON

ANEXO B – CRITÉRIOS DE PONDERAÇÃO DO CATÁLOGO DE FUNCIONALIDADES PAAS

Este anexo descreve os critérios e os pesos atribuídos aos requisitos arquiteturais avaliados pela sonda autônoma na geração do *score* global comparativo entre as plataformas PaaS:

- **WAF Integrado com Bloqueio de Injeções (Peso 5):** Requisito crítico de segurança defensiva perimetral;
- **Criptografia de Segredos em Repouso via Fernet (Peso 5):** Imprescindível para conformidade de dados e proteção de credenciais;
- **Detecção Automática de Dockerfile / GitOps (Peso 5):** Elemento central para experiência do desenvolvedor e automação de entrega;
- **Rollback Automático Instantâneo (Peso 5):** Mecanismo fundamental de resiliência e continuidade de negócios;
- **Roteamento Dinâmico com Certificados Wildcard (Peso 5):** Automação completa de infraestrutura de rede e segurança de transporte;
- **Ajuste Dinâmico de Quotas de CPU/RAM em Execução (Peso 5):** Capacidade avançada de gestão sem interrupção de contêineres;
- **Bancos de Dados Gerenciados com Persistência (Peso 5):** Suporte a dados transacionais como serviço de primeira classe;
- **Backups Automatizados em Nuvem S3 com SigV4 (Peso 5):** Estratégia mandatória de recuperação de desastres (*Disaster Recovery*);
- **Hardening de Contêiner e Isolamento de Recursos (Peso 4):** Aplicação de `no-new-privileges` e limite de processos no kernel;
- **Terminal Web com Auditoria Defensiva (Peso 4):** Prevenção contra destruição acidental ou ataques de dentro do contêiner;
- **Monitoramento de Uptime com Auto-Heal (Peso 4):** Recuperação autônoma frente a travamentos de processos.

APÊNDICE A – CATÁLOGO DOS CASOS DE TESTE E2E DA SONDA DE AUDITORIA

Este apêndice lista os 58 casos de teste ponta a ponta (E2E) projetados e validados pelo aplicativo autônomo *DeployDock Multi-Model Benchmark & PaaS Tester Probe v2.2*, detalhando as funcionalidades e as asserções de segurança aplicadas à plataforma.

1. **Módulo 1: Autenticação, RBAC e Controle de Acesso (6 Testes)**
   - `auth_login_and_token`: Fluxo OAuth2 Password Flow e emissão do token JWT assinado;
   - `auth_profile_rbac`: Validação de perfil autenticado e verificação do papel (*role*) de administrador;
   - `auth_invalid_credentials_guard`: Defesa contra credenciais incorretas com retorno HTTP 401;
   - `rbac_privilege_enforcement`: Bloqueio HTTP 403 para usuários no perfil *Viewer* em rotas administrativas;
   - `auth_totp_2fa_status`: Validação de suporte a 2FA via algoritmo TOTP (RFC 6238);
   - `auth_sessions_and_policies`: Auditoria de sessões simultâneas e expiração de tokens ativos.
2. **Módulo 2: Segurança Ativa e Web Application Firewall (10 Testes)**
   - `waf_sqli_blocking`: Bloqueio de injeção SQL clássica (`' OR '1'='1`) com retorno HTTP 403;
   - `waf_blind_sqli_sleep`: Bloqueio de injeções cegas com funções de pausa (`pg_sleep / WAITFOR`);
   - `waf_xss_filter`: Interceptação de injeção de scripts no cliente (`<script>alert(1)</script>`
   - `waf_path_traversal_linux`: Bloqueio de navegação relativa em diretórios (`../../etc/passw`
   - `waf_path_traversal_win`: Bloqueio de manipulação de caminhos em ambiente Windows (`..`
     `..`
     `windows`
     `win.ini`);
   - `waf_rce_log4shell`: Bloqueio de expressões de execução remota JNDI (`${jndi:ldap://...}`
   - `waf_reverse_shell`: Detecção de descritores de redirecionamento de shell (`bash -i >& /dev/tcp/...`);

- `anti_ssrf_git_protection`: Validação e bloqueio de URLs de repositório apontando para redes privadas locais;
- `security_headers_enforcement`: Presença dos 5 cabeçalhos estritos de proteção HTTP;
- `ip_auto_ban_brute_force`: Banimento automático de IP após ultrapassagem do limite de falhas de autenticação.

3. **Módulo 3: Ciclo de Vida de Contêineres e Terminal Seguro (8 Testes)**
   - `container_list_and_inspection`: Consulta detalhada de metadados, redes e estados de execução;
   - `container_live_cgroup_quotas`: Atualização dinâmica de cotas de memória e CPU sem reinicialização;
   - `container_logs_streaming`: Transmissão contínua de logs de execução via canal WebSocket;
   - `terminal_web_guarded`: Validação da sessão de console web interativo;
   - `terminal_block_docker_sock`: Interceptação e bloqueio de tentativas de acesso a `/var/run/docker.sock`;
   - `terminal_block_fork_bomb`: Bloqueio da sintaxe de esgotamento de processos `:(){ :|:& };:;`
   - `terminal_block_root_destroy`: Prevenção contra instruções destrutivas do sistema de arquivos (`rm -rf /`);
   - `container_hardening_inspection`: Confirmação da presença de `no-new-privileges` e limite `pids.max=256`.
4. **Módulo 4: Explorador de Arquivos e Armazenamento (6 Testes)**
   - `file_explorer_directory_tree`: Listagem recursiva e navegação pela árvore de diretórios do contêiner;
   - `file_explorer_inplace_edit`: Edição e gravação direta de arquivos de configuração em tempo real;
   - `file_explorer_upload_base64`: Transferência segura de arquivos binários e texto via carga codificada;
   - `file_explorer_download_binary`: Obtenção e validação de integridade de arquivos do contêiner;
   - `volume_tar_gz_instant_backup`: Compactação e exportação de volumes de dados em arquivo compactado;

- `volume_tar_gz_restore`: Restauração e descompactação de volumes em contêineres alvos.

5. **Módulo 5: GitOps, Roteamento e Implantação Contínua (12 Testes)**

   - `gitops_auto_dockerfile`: Detecção e compilação de projetos sem arquivo Dockerfile explícito;
   - `gitops_webhook_hmac_trigger`: Processamento de gatilhos automáticos de build via assinatura HMAC;
   - `gitops_private_repo_token`: Clonagem de repositórios privados com tokens PAT descriptografados;
   - `gitops_preview_deployments`: Provisionamento de ambientes temporários isolados para Pull Requests;
   - `gitops_healthcheck_evaluation`: Monitoramento contínuo do endpoint de saúde após implantação;
   - `gitops_automatic_rollback`: Reversão imediata para a versão anterior estável em caso de falha de inicialização;
   - `traefik_subdomain_routing`: Alocação de subdomínio dinâmico e integração com proxy reverso Traefik v3;
   - `traefik_route_collision_guard`: Prevenção contra colisão de caminhos e domínios duplicados;
   - `traefik_tls_wildcard_readiness`: Verificação de parâmetros para provimento de certificados Let's Encrypt;
   - `fernet_vault_env_encryption`: Confirmação de cifra de variáveis de ambiente no disco via AES-128;
   - `fernet_vault_runtime_injection`: Injeção transparente de segredos em variáveis de processo do contêiner;
   - `deploy_concurrency_locking`: Bloqueio de compilações simultâneas conflitantes na mesma aplicação.

6. **Módulo 6: Bancos Gerenciados, PaaS e Backups Cloud (9 Testes)**

   - `paas_postgres_managed_provisioning`: Criação automatizada de contêiner PostgreSQL 16 com volume persistente;
   - `paas_redis_managed_provisioning`: Inicialização de instância gerenciada de cache em memória Redis;
   - `paas_mariadb_managed_provisioning`: Inicialização de instância gerenciada MariaDB/MySQL;

- `paas_mongodb_managed_provisioning`: Inicialização de banco de documentos NoSQL MongoDB;
- `paas_one_click_catalog`: Instanciação de modelos prontos da biblioteca (n8n, MinIO, Grafana);
- `paas_s3_backup_sigv4`: Realização de backup programado para nuvem compatível com AWS S3 SigV4;
- `paas_cloudflare_r2_minio_support`: Teste de compatibilidade de credenciais para Cloudflare R2 e MinIO;
- `paas_cron_scheduler_app`: Agendamento de tarefas periódicas automatizadas por aplicação;
- `paas_failure_heuristic_diagnosis`: Motor de análise de erros de build com recomendação de correção.

7. **Módulo 7: Observabilidade, Telemetria e Notificações (7 Testes)**

- `telemetry_cpu_ram_disk_metrics`: Coleta em tempo real de estatísticas de consumo do hospedeiro;
- `telemetry_system_prune_cleanup`: Expurgo seguro de contêineres e imagens não utilizadas no Docker;
- `uptime_monitor_http_health`: Monitoramento em tempo real da disponibilidade e do tempo de resposta de URLs;
- `uptime_auto_heal_restart`: Reinicialização inteligente do contêiner quando o serviço HTTP atinge limite de falhas;
- `notifications_multichannel_dispatch`: Envio de mensagens de alerta integradas (Discord, Slack, Telegram, SMTP);
- `notifications_history_log`: Registro persistente do histórico de notificações emitidas;
- `prometheus_metrics_endpoint`: Disponibilização de métricas do sistema no padrão de texto Prometheus.